

Intelligent for Embedded Systems (ET -8305)

*Department of Electronics and Telecommunication Engineering
Mbeya University of Science and Technology*



Email : [dsinkonde2013@gmail](mailto:dsinkonde2013@gmail.com)



Expected Outcomes

Students can expect to gain the following:

- ✓ Analyze the data acquisition systems
- ✓ Be able to explain the whole idea of instrumentation systems
- ✓ Use software tools (e.g. MATLAB) to simulate and analyse the embedded systems

Assignments and Learning Evaluation

- Mode of Assessment
 - Continuous Assessment (CA) 40%

Assignments

- Total Score = 10% , End of submission on the 6th Week
 - 05th December, 2023

▪ **Tests**

- Test to be conducted Starting From Week 8 = 30%
 - 19th December, 2023
- University Examination (UE) 60%
- Number of Credits : 9

Assignments

- Project
 - a) Analysis of the literature
 - b) Design of a solution
 - c) Development of the designed solution
 - d) Experimental Analysis
 - e) Presentation Skills
- Different combinations for a, b, c, d, e but $a+b+c+d+e=15\%$
- Up to 5 people
- Knowledge of **MATLAB**
- Topics available at the end of the slide (now :-)

General Course Information

Textbooks / References

1. Barney G C, “Intelligent Instrumentation: Micro processor application in measurement and control,” Prentice Hall, Engle Wood Cliff NJ, 2001.
2. H.S. Store, “Micro Computer Interfacing,” Addison Wesley.
3. G. M. Johnson “LAB view Graphical Programming,” TMH.
4. Barry Paton, “Sensor, Transducers and Labview”, Prentice Hall.

General Course Information

Topic to be Covered

Introduction:

Intelligent versus Dumb instruments, A historical perspective of instrumentation systems. Review of digital transducers. Interfacing micro computers computer port to high power devices. Optical shot encoder communication standards. Concepts of Real Time system and its application

Introduction to Embedded Systems: Review of microcontrollers and Digital Signal Processors (DSP), architecture, peripheral modules. Embedded micro controller cores (ARM, RISC, CISC, SOC), addressing modes, interrupts structure, hardware multiplier, pipelining. Hardware/Software co-design. Architecture of embedded systems.

Topic to be Covered

Intelligent devices: Use of intelligent devices to reduce environmental disturbance effects, features and benefits, communication and computation in intelligent devices, future trends

Data Acquisition: Details of Data Acquisition systems (DAS) Logic control systems, Continuous & Batch modes, Single and multi loop controller. Details of Data logger and its application

Virtual Instrument: Architecture of Virtual instrument and its relation to operating system. Software overview: LABVIEW, Graphical User Interface (GUI), Control and indicators: G programming- Data type, Data flow programming editing and running a virtual instrument.

Topic to be Covered

G Programming: G Programming details in LABVIEW, G Programming tools and libraries. Programming structure: For loop, while loop. CASE structure, Sequence Structure arrays and clusters. Array operations- Bundle/Unbundled String and file I/O. High level and low level I/Os. Attribute nodes, Local and global variables

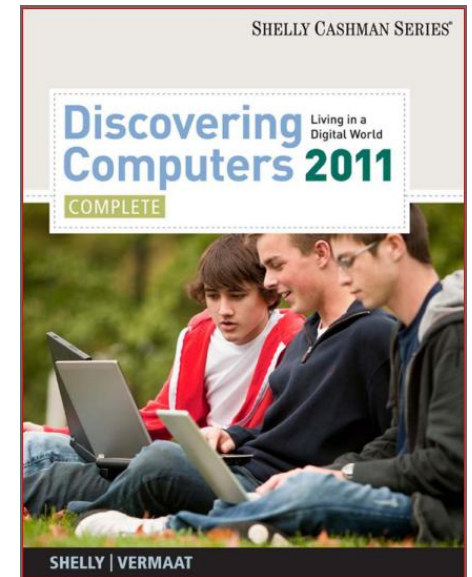
Software Development for instrumentation: Software development for Temperature (Low and High) level, Speed, pressure etc.

Embedded Software Development: Assemblers, linkers and loaders. Binary file formats for processor executable files. Typical structure of timer-interrupt driven programs. GNU-GCC compiler introduction, programming with Linux environment and gnu debugging, gnu insight with step level trace debugging, make file interaction, building and execution.

Topic to be Covered

Design with ARM Processor: Introduction to ARM instruction set, addressing modes, operating modes with ARM core, ARMTDMI modes, ADC, Timers, Interrupt structure. Byte ordering (LE, BE), Thumb mode normal mode instructions changes, Pipeline utilization with all register allocations. Compare with ARM7, ARM9, and ARM11 with new features additions. System design with ARM processor.

Lecture 1:

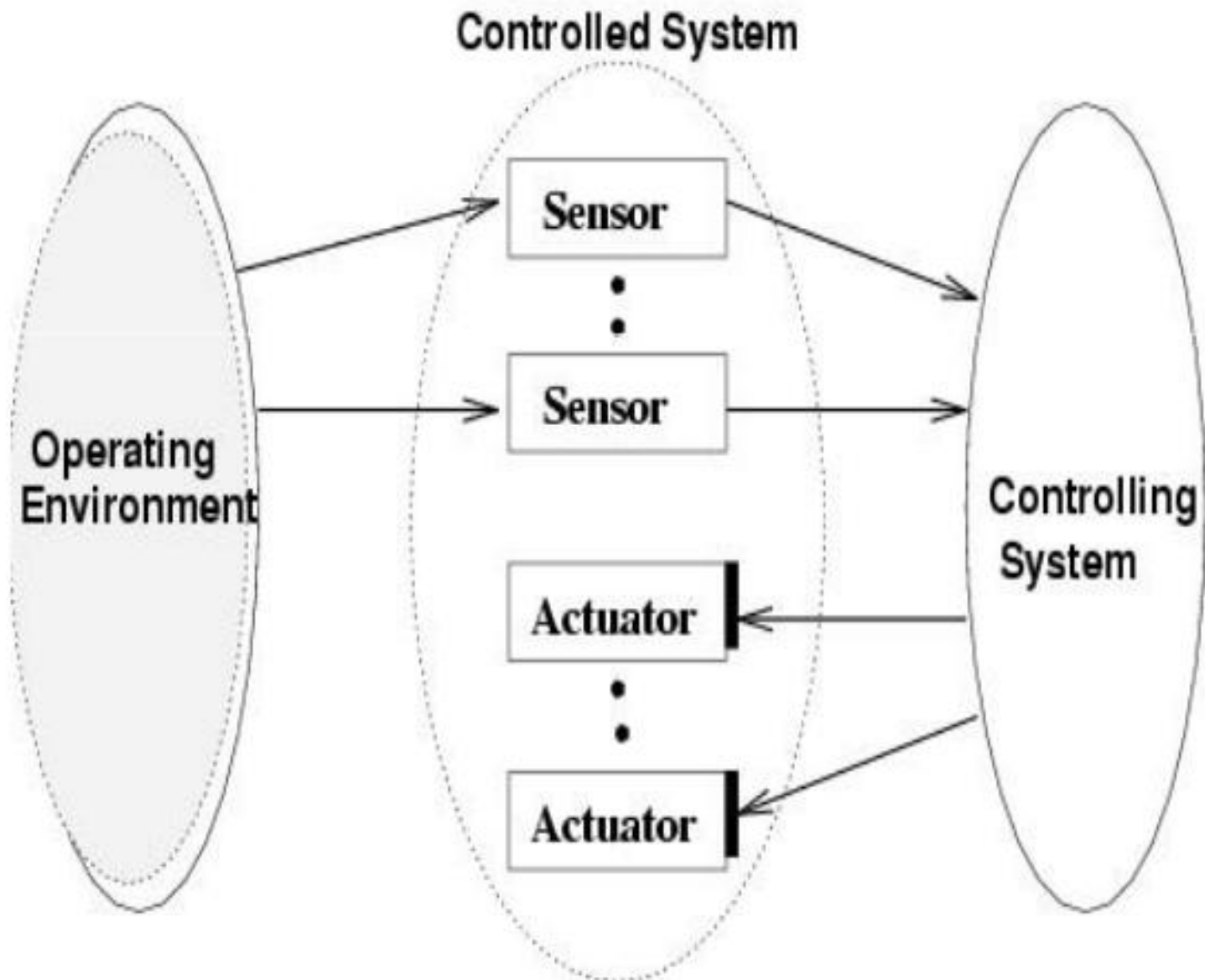


Real time systems concepts

Overview of Real-Time Systems

- The differences between real time computing systems and general purpose computing systems are very profound.
- We will examine many of these differences in the upcoming slides.
- A **real time computing system** is one that requires that correct results be produced within specified **deadline** periods. (Deadline is a key word)
- Results produced after a specific time period has elapsed may well be of **absolutely no value** and may mean loss of life or an aircraft crash; yet other failures might **not** be **quite as disastrous** if real time system is not quite as responsive.
- Real time:
 - Run on wide range of computer hardware;
 - Used in many different kinds of applications.
- Some real time systems are **embedded** in aircraft instrumentation:
 - your microwave, cell phone, cruise control, and hosts of other applications.
 - Are often part of a larger system;
 - Oftentimes their presence is not obvious to a user.

Typical RTS



Example: Car

- **Mission:** Reaching the destination safely.
- **Controlled System:** Car.
- **Operating environment:** Road conditions and other cars.
- **Controlling System**
 - *Human driver:* Sensors - Eyes and Ears of the driver.
 - *Computer:* Sensors - Cameras, Infrared receiver, and Laser telemeter.
- **Controls:** Accelerator, Steering wheel, Break-pedal.
- **Actuators:** Wheels, Engines, and Brakes.

Definitions -1

- **System:** It is an arrangement in which all its units, assembled work together according to set of rules. E.g., Watch,
- **Response time:** time between presentation of a set of inputs and the appearance of the corresponding outputs.
- **Events:** Change of state causing a change of flow- of- control of a computer program.
- **Reactive:** system 'reacts' to environmental changes (temperature changes).
- **Embedded:** It is a combination of hardware and software, it is designed to perform a particular task that has to be completed within a given time.

Definitions -2

- **synchronous:** events occur at predictable times in the flow- of- control.
- **asynchronous:** unpredictable (interrupts!).
- state- based vs. event- based:
 - plane wing is at an angle of 32° (state)
 - plane wing moved up 4° (event)
- **deterministic system:** for each possible state and each set of inputs, a unique set of outputs and next state of the system can be determined.

Definitions - 3

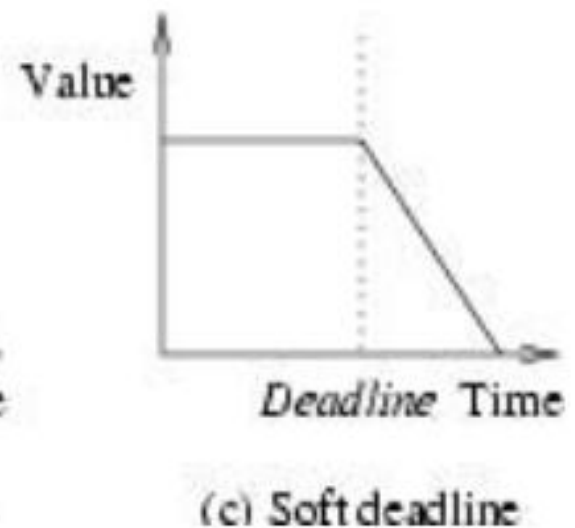
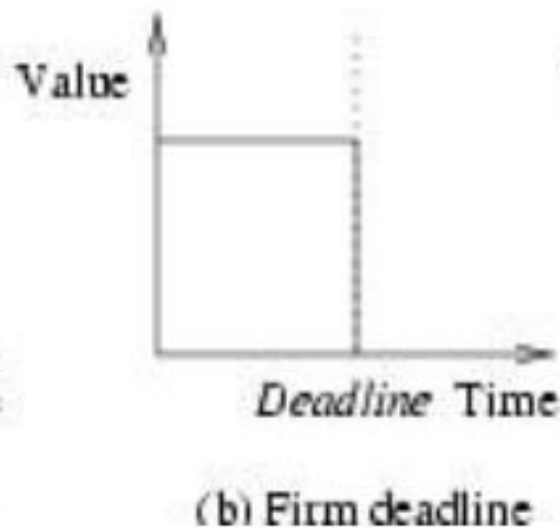
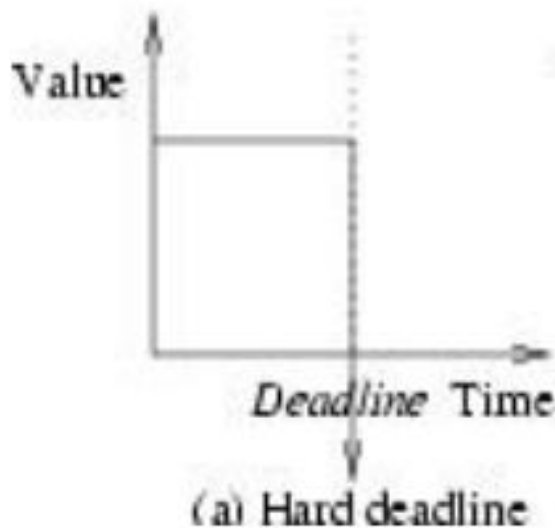
- **Utilization:** measure of 'useful' work a system performs.
- **RTS:** Correctness depends on results PLUS the time of delivery! Failure can have severe consequences.
- What are real- time systems? Planes, cars, washer, video player, thermostat, video games, weapons,...
- **Related:** QoS management, resource management, adaptive systems, embedded systems, pervasive and ubiquitous computing,...

Definitions - 4

- A **real-time system** requires that results be produced within a specified deadline period
- An embedded system is a computing device that is part of a larger system (I.e. automobile, airliner)
- A **safety-critical system** is a real-time system with catastrophic results in case of failure
- A hard real-time system guarantees that real-time tasks be completed within their required deadlines
- A **soft real-time system** provides priority of real-time tasks over non real-time tasks

Hard versus Soft

- **HARD:** miss a deadline and you're in trouble!
(planes, trains, factory control, nuclear facilities,...)
- **SOFT:** try to meet deadlines, but if not, systems still work, although with degraded performance
(multimedia, thermostat,...)
- **FIRM:** late results are worthless, but you are not in trouble



Example: cruise control

- Regulates speed of car by adjusting the throttle: driver sets a speed and car maintains it.
- Measures speed through device connected to drive shaft.
- Hard real-time: drive shaft revolution events.
- Soft real-time: driver inputs, throttle adjustments.



More examples

- cars: engine control, ABS, drive-by-wire
- planes: stability, jet engine, fly-by-wire
- computers: peripherals, applications
- military: weapons, satellites
- domestic: microwave, thermostat, dishwasher
- medical: pacemaker, medical monitoring
- protection: intruder alarm, smoke/gas detection

System Characteristics

- Single purpose
- Small size
- Inexpensively mass-produced
- Specific timing requirements

Characteristics -1

■ Single Purpose :

- Single purpose is typical: controlling anti-lock brakes; toaster, cell phone.
- This makes the operating system **simple** too, as many characteristics integral to general – purpose operating systems are not available or needed.

■ Size:

- Often found in severely cramped space – but sufficient for operations.
 - ▶ Examples: wrist watches, cell phones, toys
 - ▶ Thus, CPU processing power is minimal
 - ▶ Amount of primary memory is also minimal.
 - ▶ Architectures: Compare 32 / 64-bit architectures with 8/16 bit processors.
 - ▶ Architecture: Compare several gigs of memory with < 1 MB memory.

Characteristics -2

■ Cost

- Typically **mass produced**: as in microwave ovens, thermostats.
- Thus, real time microprocessors are often inexpensive
- Organization of real time systems are designed to minimize cost:
 - ▶ To eliminate bus architectures, the physical organization for embedded controllers are often organized as a **system-on-a-chip** , which has all necessary interconnections.
 - ▶ Chip includes memory, cache, a MMU (for possible address translation), any peripheral ports necessary - **all in a single integrated circuit**.
 - ▶ Such organization is typically much less expensive than typical bus-oriented architectures.

Characteristics - 3

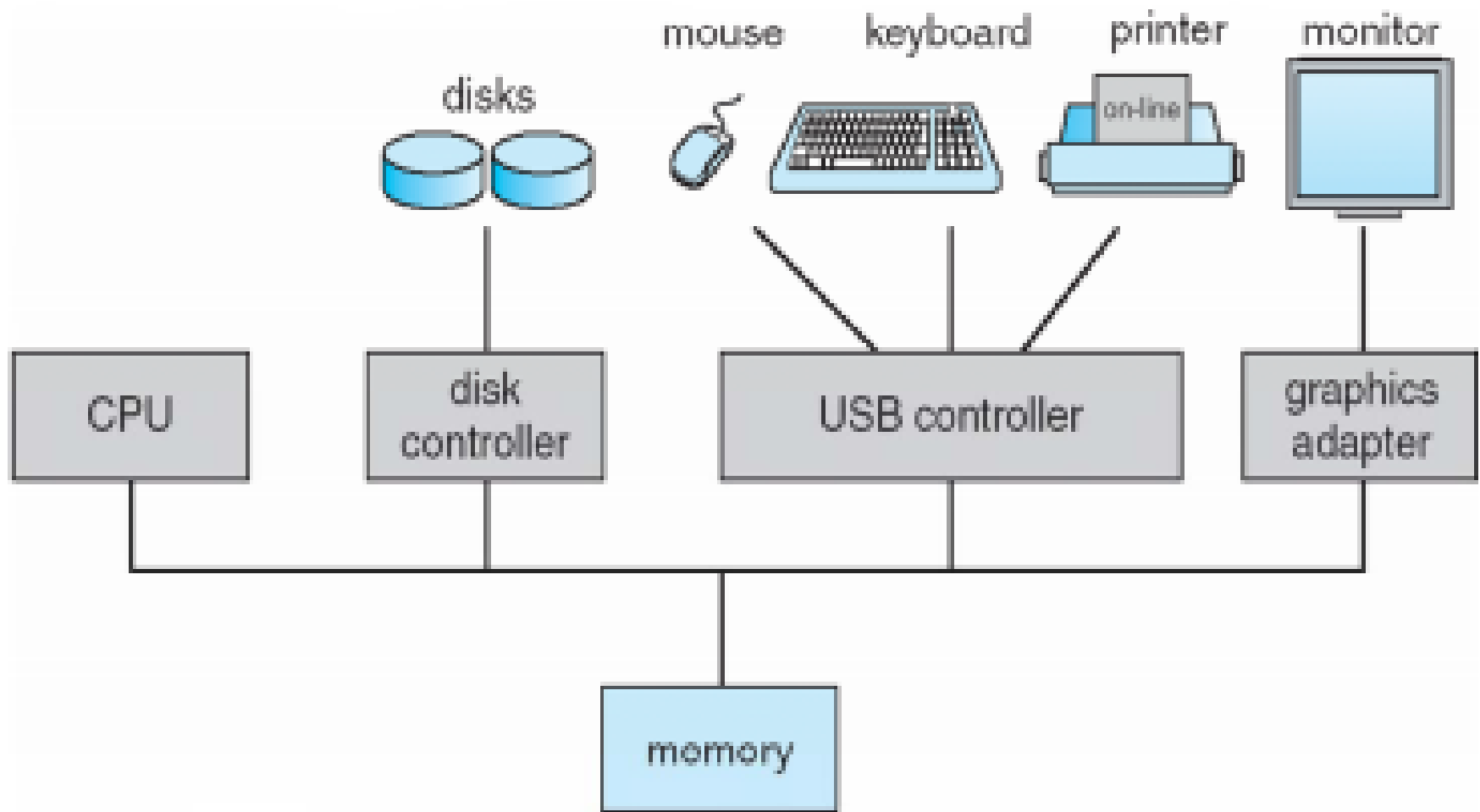
■ Timing :

- This is the feature that **impacts almost everything else** and makes real time systems what they really are!
- Both 'hard' and 'soft' real time systems have timing requirements.
- So, we will need to develop **real time scheduling algorithms** that provide priority to the highest scheduled processes.
- **Schedulers absolutely must ensure** that the priority of real time tasks does not degrade over time.
- So, we must minimize the response time to interrupts – recognize the interrupt, save context, transfer control to the handler, etc... as one can easily imagine.

System-on-a-Chip

- Many real-time systems are designed using system-on-a-chip (SOC) strategy
- SOC allows the CPU, memory, memory-management unit, and attached peripheral ports (I.e. USB) to be contained in a single integrated circuit

Bus-Oriented System



Features of Real-Time Kernels

- Most real-time systems do not provide features found in desktop systems.
- Simply not needed.
- Do not need (in general) support for
 - A variety of peripheral devices – graphical displays, CD, DVD drives...
 - Protection and security mechanism
 - Support for multiple users...
 - Note: Windows XP has > 40,000,000 lines of code; a typical real-time operating system usually is written in **thousands of source code lines** !
- Reasons include
 - Real-time systems are typically single-purpose.
 - Real-time systems often do not require interfacing with a user.
 - General-purpose features often require more substantial hardware than that found in a RT system.

Virtual Memory in Real-Time Systems

(1) **Real-addressing mode** where programs generate actual addresses.

- But there is no memory protection between processes.
- We might also need to specify exactly where in memory the program is to be loaded.
- But the speed is very difficult to beat!
- No time spent in address translations.

Very commonly found in real time systems having **hard, real-time constraints**.

Some real time operating systems running on microprocessors containing a MMU, actually disable the MMU to gain performance benefit in referencing physical addresses directly.

(2) **Relocation register mode** .

- In this scheme, we have a relocation register is set to the process' load point.
- Physical addresses are added to contents of the relation register formed by adding logical (L) to R (relocatable) to form P (Physical) addresses..
- But here again, there is no protection between processes.

(3) **Implementing full virtual memory** .

- But here we may have page tables and translation look-aside buffers.
- While this strategy does indeed provide memory protection, it is costly...



Address Translation

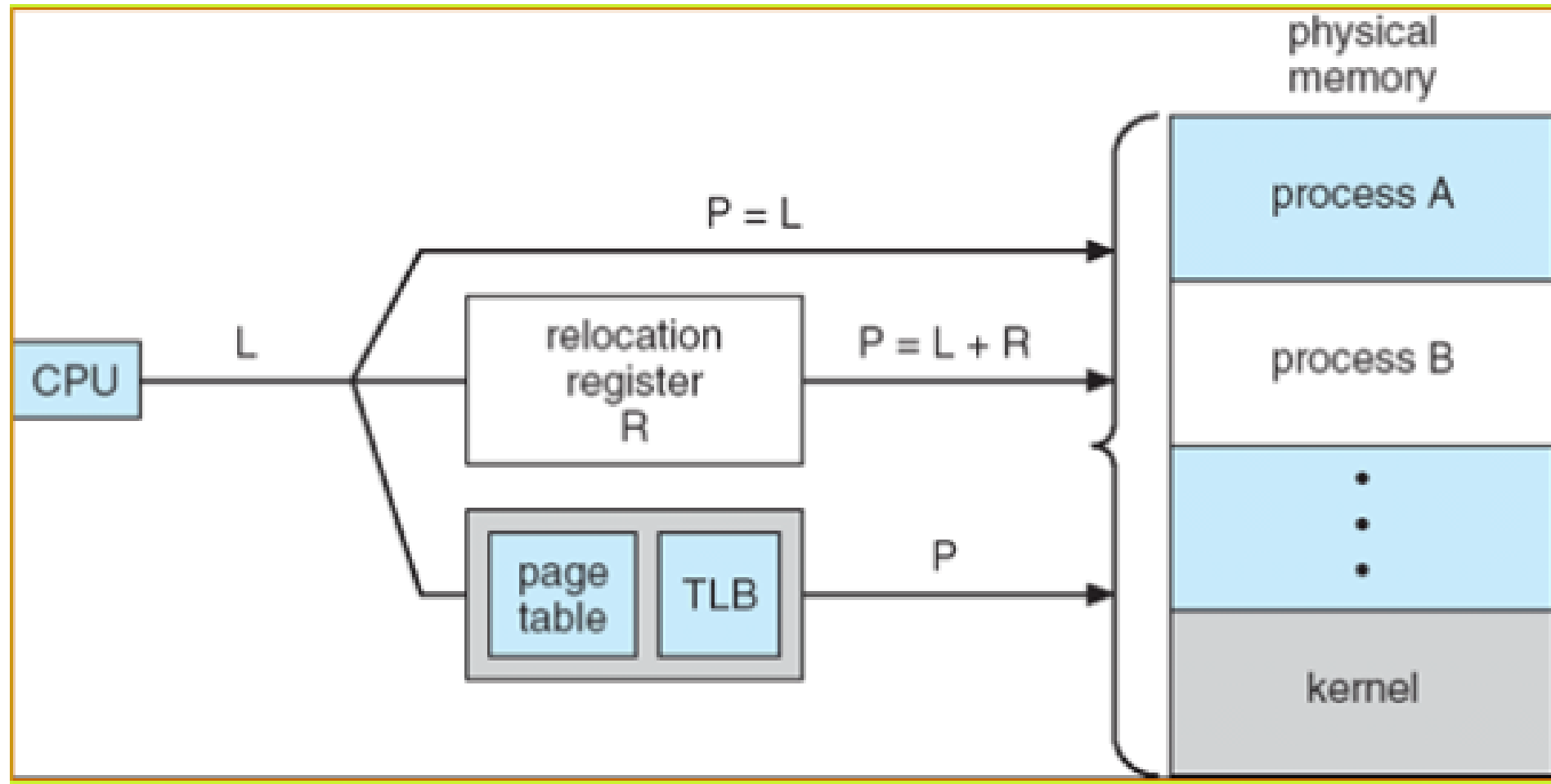


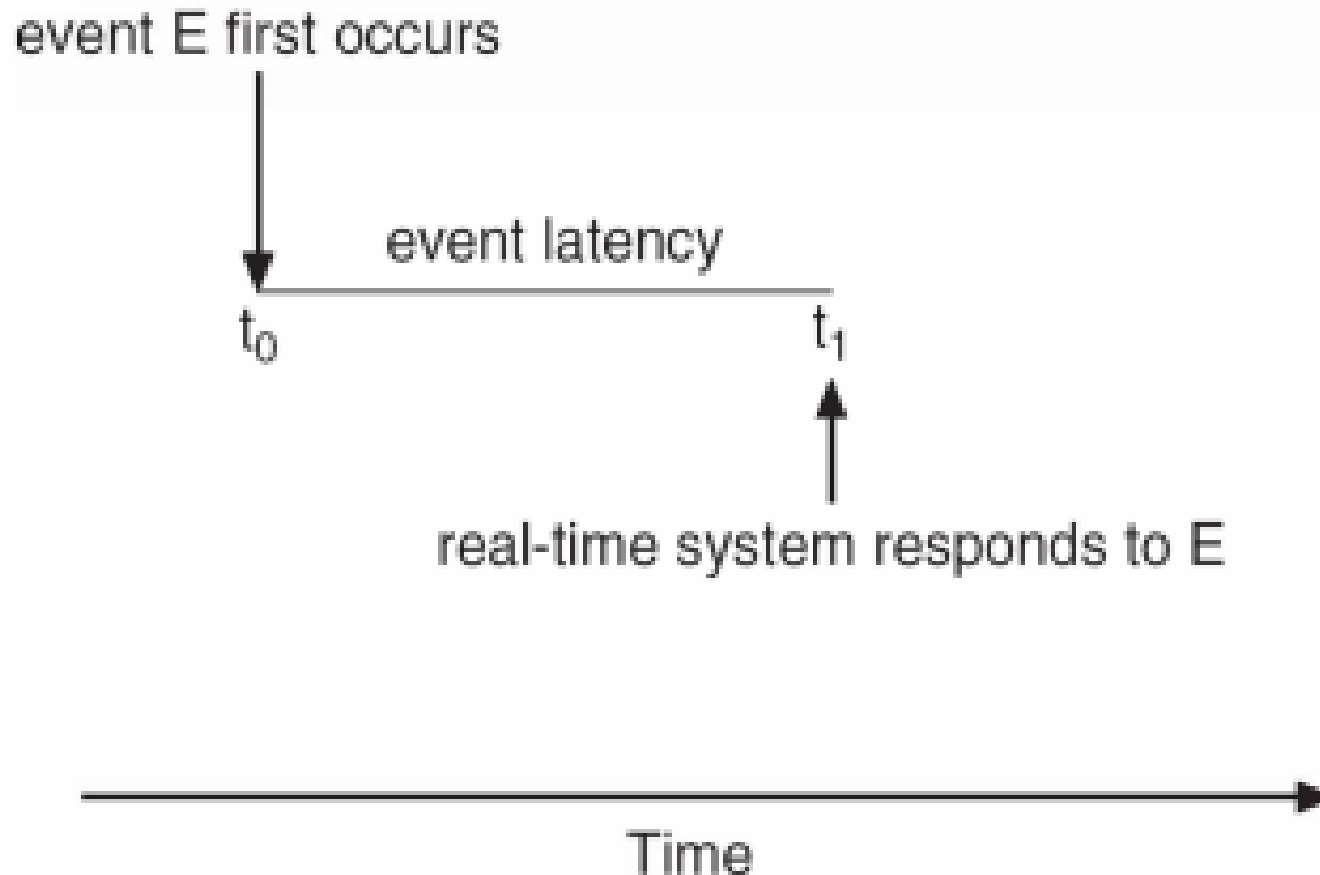
Diagram of the previous three memory access techniques

Implementing Real-Time Systems

- In general, real-time operating systems must provide:
 - (1) Preemptive, priority-based scheduling
 - (2) Preemptive kernels
 - (3) Latency must be minimized

Minimizing Latency

- **Event latency** is the amount of time from when an event occurs to when it is serviced.

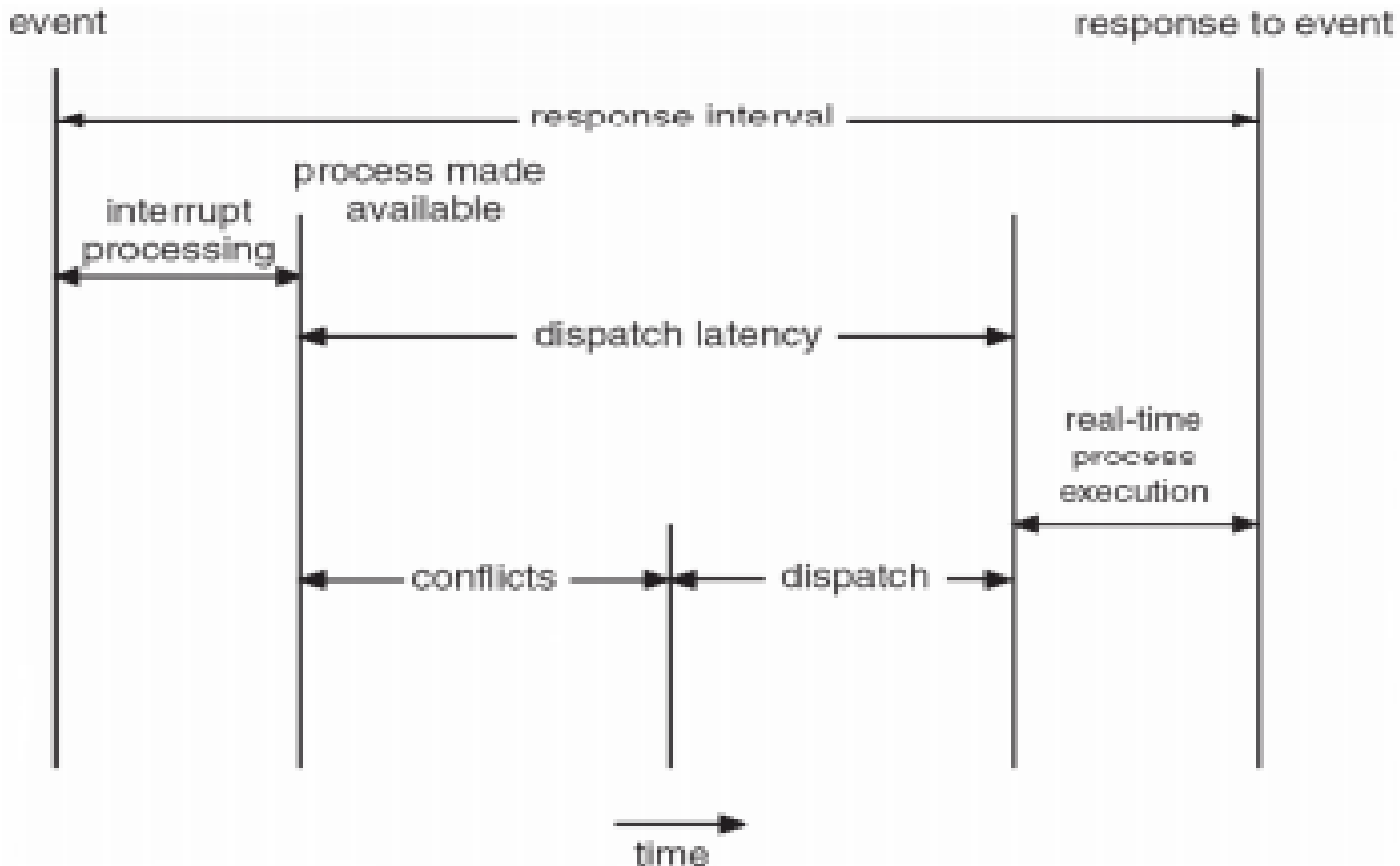


Minimizing Latency

- But all events are not created equal.
- Different events have different latency times.
- Some have a very short latency time; others longer.
 - **Example:** an embedded system controlling something like a toaster or some kind of radar, might tolerate a latency of several seconds.
- In RT systems we've got two types of latencies to consider:
 - Interrupt latency, and
 - Dispatch latency.

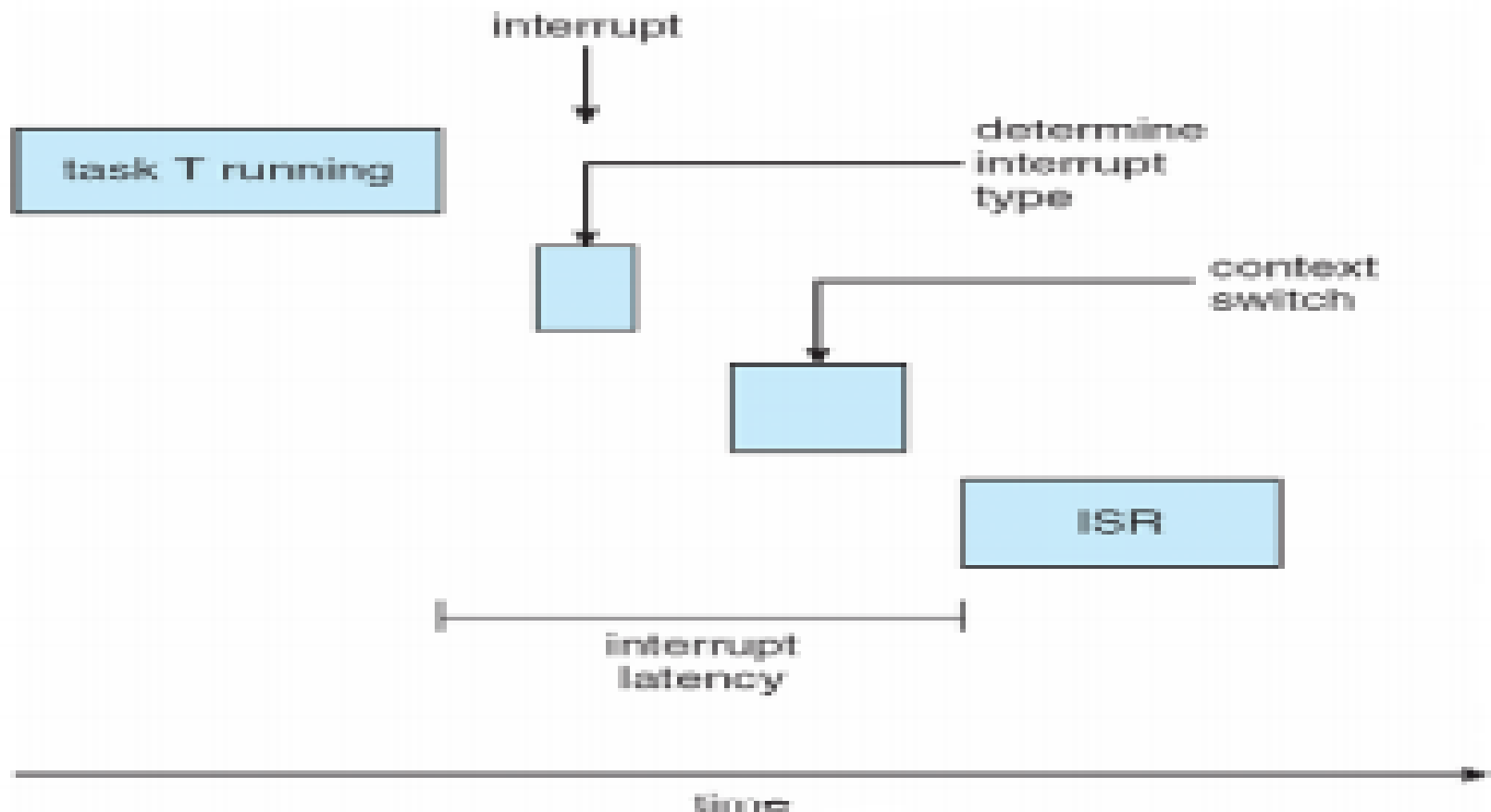
Dispatch Latency

- Dispatch latency is the amount of time required for the scheduler to **stop** one process and **start** another



Interrupt Latency

- **Interrupt latency** is the period of time from when an interrupt arrives at the CPU to when it is serviced to when it is serviced



Questions



End of segment